

UNIVERSITÀ DEGLI STUDI DI MESSINA
CORSO DI LAUREA IN INFORMATICA

SecureChat: chat multiutente TCP

Massimiliano Spartà e Simone Adamo

Matricole 566093 e 567317

ANNO ACCADEMICO 2025–2026

INDICE

1	SecureChat	2
1.1	Introduzione	2
1.2	Stato dell'arte	2
1.3	Definizione del problema	3
1.4	Metodologie utilizzate	4
1.4.1	Architettura del progetto	4
1.4.2	Creazione del server TCP	4
1.4.3	select() e multiplexing I/O	5
1.4.4	epoll(): l'alternativa moderna	6
1.4.5	Risoluzione del problema	7
1.4.6	Un client dedicato: chat_client.c	10
1.4.7	Cifratura del canale con TLS	11
1.5	Presentazione dei dati	14
1.6	Discussione dei risultati	15

1 SECURECHAT

1.1 Introduzione

Il progetto discusso in questa relazione corrisponde alla traccia #1 (*Secure Chat*) tra quelle proposte dal docente per il corso di Laboratorio di Reti e Sistemi Distribuiti Mod. B, sviluppato in coppia da Simone Adamo e Massimiliano Spartà.

L'obiettivo è realizzare un server di chat testuale centralizzato in grado di gestire più client TCP contemporaneamente, ridistribuendo i messaggi ricevuti a tutti gli utenti connessi nella stessa stanza. Il vincolo architetturale imposto dalla consegna è che questa concorrenza venga gestita **in un singolo thread**, tramite `select()` o `epoll()`, e non tramite un thread (o processo) per ogni connessione.

Rispetto al minimo richiesto, il progetto è stato esteso in quattro direzioni:

- sono state realizzate **entrambe** le varianti di multiplexing (`select()` ed `epoll()`), condividendo la stessa identica logica applicativa, per poterle confrontare direttamente sullo stesso protocollo;
- è stato affrontato un problema spesso trascurato nelle implementazioni didattiche di protocolli su TCP: il *framing* dei messaggi, cioè la corretta ricostruzione dei confini tra un messaggio applicativo e l'altro a partire da un flusso di byte che non li rispetta;
- è stato scritto un client dedicato, `chat_client.c`, per poter testare il protocollo end-to-end senza dover ricorrere unicamente a `netcat`, che non impone alcuna disciplina sull'invio delle righe e non mette quindi alla prova un client reale;
- il canale è stato cifrato con TLS (OpenSSL), così che il nome *SecureChat* corrisponda a una proprietà effettiva del sistema e non solo al titolo del progetto.

1.2 Stato dell'arte

Il problema della gestione di molte connessioni simultanee su un server di rete è storicamente noto come *problema C10K* (la capacità di gestire

diecimila connessioni concorrenti su una singola macchina), e ammette essenzialmente tre approcci architetturali.

Thread (o processo) per connessione. È l'approccio più semplice da programmare: ogni nuova connessione riceve un thread dedicato che può usare chiamate bloccanti (`recv()`, `send()`) senza preoccuparsi delle altre connessioni. Lo svantaggio è che ogni thread occupa uno stack (tipicamente qualche MB) e il sistema operativo deve schedarli tutti: con migliaia di connessioni il consumo di memoria e l'overhead di context-switching diventano proibitivi.

I/O multiplexing con `select()`. Un singolo thread tiene traccia di un insieme di file descriptor e si blocca su `select()` finché almeno uno di essi non è pronto per la lettura o la scrittura. Nessun thread aggiuntivo è necessario; il limite pratico è `FD_SETSIZE` (tipicamente 1024) e il fatto che, ad ogni ciclo, il kernel debba scandire linearmente tutti i descriptor monitorati per capire quali sono pronti: complessità $O(N)$ nel numero di connessioni.

I/O multiplexing con `epoll()`. Elimina il limite di `select()` spostando nel kernel Linux lo stato dei file descriptor monitorati: `epoll_wait()` restituisce direttamente la lista dei descriptor pronti, con complessità $O(1)$ rispetto al numero di eventi pronti, indipendentemente da quanti file descriptor sono monitorati in totale. È il meccanismo su cui si basano i server event-driven moderni (Nginx, e in generale i motori come `libevent/libuv`).

Per il progetto si è scelto di implementare e confrontare direttamente le ultime due soluzioni, scartando la prima perché esplicitamente esclusa dalla consegna.

1.3 Definizione del problema

Il testo guida della traccia richiede:

Una chat multi-utente in cui un server centrale riceve messaggi da N client e li ridistribuisce a tutti gli altri. Il server usa `select()` o `epoll()` per gestire connessioni multiple in un singolo thread, senza bloccarsi su nessun client. I client possono organizzarsi in stanze indipendenti; il broadcast avviene solo all'interno della stessa stanza.

Da cui derivano i requisiti concreti:

- comunicazione tramite protocollo TCP;
- gestione contemporanea di più connessioni, in un singolo thread di rete;
- identificazione degli utenti tramite nickname;
- supporto a stanze indipendenti, con broadcast filtrato per stanza.

1.4 Metodologie utilizzate

1.4.1 Architettura del progetto

Il codice è stato suddiviso in più file e cartelle per isolare la logica applicativa dal meccanismo di multiplexing:

```

1 Progetto_Laboratorio/
2 |
3 +-- include/
4 |   '-- chat.h    # struttura ClientInfo, costanti, prototipi
5 |   '-- tls.h     # contesti SSL lato server/client
6 |
7 +-- src/
8 |   +-- registry.c # nickname, stanze, framing, comandi
9 |   +-- main_select.c # main loop del server select()
10 |   +-- main_epoll.c # main loop del server epoll()
11 |   '-- chat_client.c # client interattivo dedicato
12 |   '-- tls.c        # configurazione OpenSSL
13 |
14 +-- build/          # file oggetto (.o)
15 +-- bin/            # eseguibili finali
16 +-- Makefile
17 '-- README.md

```

Isolare la logica applicativa in `registry.c` è stata una scelta deliberata: permette di verificare che le due varianti si comportino in modo identico, perché la differenza tra loro riguarda *esclusivamente* il meccanismo con cui il server scopre quali socket sono pronte, non cosa fa una volta che lo sono. Lo stesso principio ha guidato l'introduzione di `tls.h/tls.c`: la parte *amministrativa* di OpenSSL (creazione del contesto, caricamento di certificato e chiave) è isolata dai main loop, che si limitano a invocarla.

1.4.2 Creazione del server TCP

Entrambe le varianti condividono la stessa sequenza di inizializzazione:

```

1  int server_fd = socket(AF_INET, SOCK_STREAM, 0);
2  setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(
    opt));
3
4  struct sockaddr_in addr = {
5      .sin_family = AF_INET,
6      .sin_port = htons(PORT),
7      .sin_addr.s_addr = INADDR_ANY,
8  };
9  bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
10 listen(server_fd, 16);

```

SO_REUSEADDR evita l'errore "Address already in use" quando si riavvia rapidamente il server durante i test (la porta resta altrimenti in stato TIME_WAIT per un certo periodo dopo la chiusura).

1.4.3 select() e multiplexing I/O

```

1  fd_set master_set, read_fds;
2  FD_ZERO(&master_set);
3  FD_SET(server_fd, &master_set);
4  int max_fd = server_fd;
5
6  while (1) {
7      read_fds = master_set;      /* select() la sovrascrive */
8      select(max_fd + 1, &read_fds, NULL, NULL, NULL);
9
10     for (int fd = 0; fd <= max_fd; fd++)
11         if (FD_ISSET(fd, &read_fds)) { /* fd pronto */ }
12 }

```

master_set contiene tutte le socket attive; read_fds è la copia temporanea che select() sovrascrive lasciando solo i descriptor pronti (da cui la necessità di ricopiarla da master_set ad ogni iterazione).

Busy-waiting vs I/O multiplexing

A differenza del *busy-waiting* (Figura 1.1), sia select() che epoll() bloccano il processo finché non è disponibile almeno un evento, azzerando lo spreco di CPU nei periodi di inattività. La differenza tra i due meccanismi riguarda invece cosa succede al risveglio: select() sa solo che almeno un file descriptor tra quelli monitorati è pronto, e deve quindi scandire linearmente l'intero insieme con un ciclo for per scoprire quale, costo $O(N)$ nel numero di fd monitorati, mentre epoll_wait() restituisce direttamente la lista dei soli fd pronti, senza bisogno di alcuna scansione.

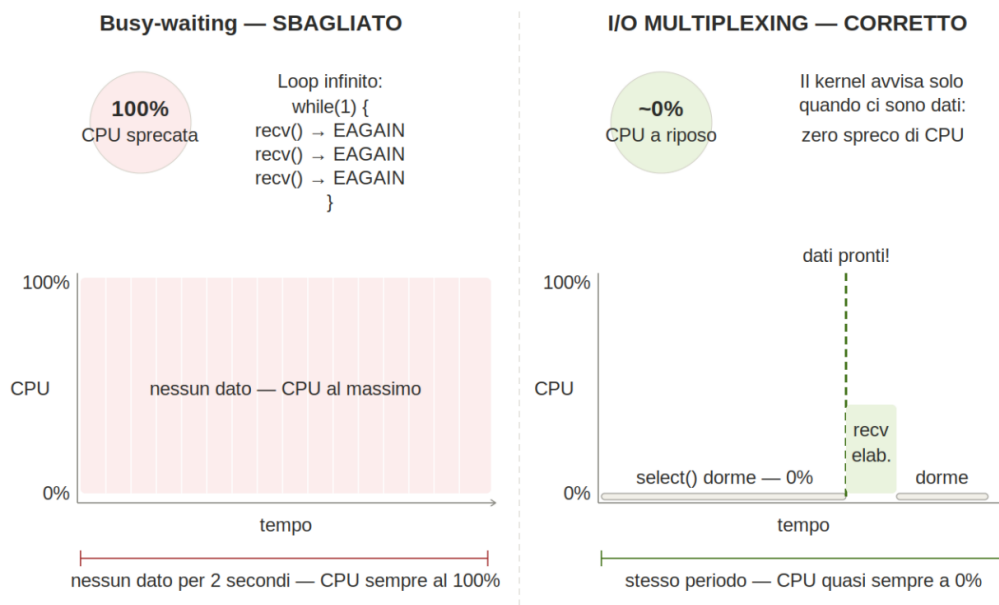


Figura 1.1: Il problema del busy-waiting

1.4.4 `epoll()`: l'alternativa moderna

```

1  int epfd = epoll_create1(0);
2  struct epoll_event ev = {.events = EPOLLIN, .data.fd =
    server_fd};
3  epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);
4
5  struct epoll_event events[MAX_EVENTS];
6  while (1) {
7      int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
8      for (int i = 0; i < n; i++) {
9          int fd = events[i].data.fd;  /* solo i fd pronti */
10     }
11 }
```

La differenza sostanziale è che l'insieme dei descriptor monitorati è mantenuto dal kernel (`epoll_ctl`), e `epoll_wait()` restituisce *solo* i descriptor effettivamente pronti: non c'è alcuna ricopiatura di insiemi né alcuna scansione di descriptor inattivi.

Alberi Rosso-Nero

L'albero rosso-nero non è usato per la `epoll_wait()` stessa, ma per gestire l'*interest list*, cioè l'insieme dei descriptor registrati tramite `epoll_ctl()`: la

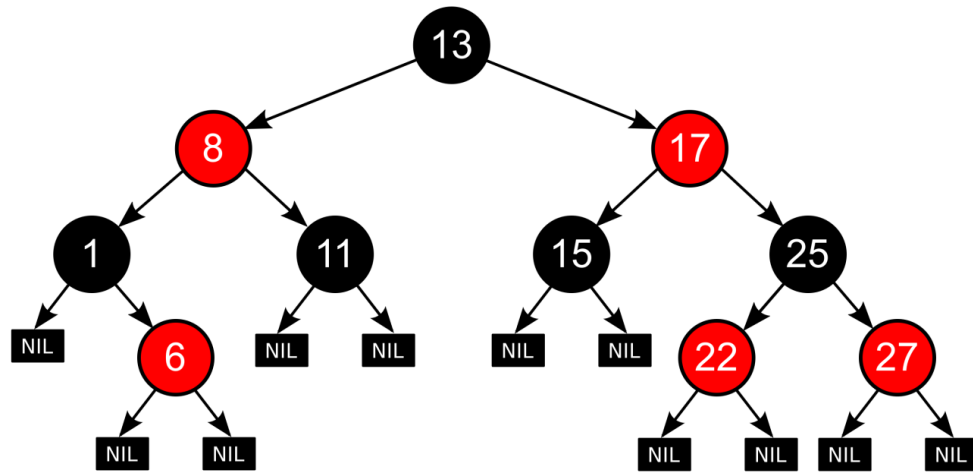


Figura 1.2: Struttura dati albero rosso-nero

ricerca, l'inserimento e la rimozione di un file descriptor in questo albero costano $O(\log n)$ nel numero di fd monitorati. I fd effettivamente pronti vengono invece accumulati in una lista separata (*ready list*) popolata da callback asincrone, ed è questa struttura a garantire che `epoll_wait()` restituisca gli eventi pronti in tempo proporzionale solo al loro numero. Un albero rosso-nero, per essere tale, deve soddisfare le seguenti proprietà:

1. Ogni nodo è rosso o nero.
2. La radice è nera.
3. Ogni foglia (NIL) è nera.
4. Se un nodo è rosso, allora entrambi i suoi figli sono neri.
5. Per ogni nodo, tutti i cammini semplici che vanno dal nodo alle foglie sue discendenti contengono lo stesso numero di nodi neri.

1.4.5 Risoluzione del problema

Struttura dati ClientInfo

```

1 typedef struct {
2     int     fd;
3     SSL     *ssl;
4     char     nickname[MAX_NAME];

```



```

5     char    room[MAX_ROOM];
6     int     active;
7     char    inbuf[MAX_LINE];    /* buffer di accumulo per il
8                                   framing */
9     size_t  inlen;
10 } ClientInfo;
11
12 ClientInfo clients[MAX_CLIENTS]; /* indicizzato per fd */

```

I campi `fd`, `nickname`, `room` e `active` rispondono direttamente ai requisiti della consegna. Il campo `inbuf/inlen` è un'aggiunta necessaria, discussa nel paragrafo seguente; il campo `ssl` è la sessione TLS associata al file descriptor, introdotta con la cifratura del canale (Sezione 1.4.7) e usata da `SSL_read()/SSL_write()` al posto di `recv()/send()` diretti sul socket.

Framing dei messaggi: il problema dei confini TCP

TCP espone un'astrazione di *flusso di byte*, non di messaggi. Una singola `recv()` può quindi restituire un messaggio applicativo incompleto, un messaggio completo seguito da parte del successivo, oppure più messaggi in una sola chiamata: la corrispondenza uno-a-uno tra una `send()` lato client e una `recv()` lato server **non è garantita dal protocollo**. Trattare ogni `recv()` come se restituisse esattamente un messaggio (approccio adottato nelle prime versioni dimostrative viste a lezione) funziona quasi sempre con client interattivi lenti come `netcat`, ma non è corretto in generale.

La soluzione adottata accumula i byte ricevuti in `inbuf` finché non si incontra un carattere `'\n'`, che segna la fine di un messaggio applicativo:

```

1 void feed_bytes(ClientInfo *c, const char *data, size_t n)
2 {
3     for (size_t i = 0; i < n; i++) {
4         char ch = data[i];
5         if (ch == '\n') {
6             c->inbuf[c->inlen] = '\0';
7             handle_line(c, c->inbuf);
8             c->inlen = 0;
9             continue;
10        }
11        if (c->inlen < MAX_LINE - 1)
12            c->inbuf[c->inlen++] = ch;
13    }
14 }

```

Il resto del buffer, se un messaggio è arrivato solo parzialmente, resta in attesa dei byte successivi invece di essere interpretato prematuramente. Questa funzione è stata verificata inviando artificialmente un messaggio

diviso in due `send()` separate (si veda la Sezione 1.5): il server lo ricompone correttamente in un'unica riga.

Comandi /nick e /join

```
1  if (strcmp(line, "/nick ", 6) == 0) {
2      strncpy(c->nickname, line + 6, MAX_NAME - 1);
3      c->nickname[MAX_NAME - 1] = '\0';
4      /* ack testuale al client */
5  } else if (strcmp(line, "/join ", 6) == 0) {
6      strncpy(c->room, line + 6, MAX_ROOM - 1);
7      c->room[MAX_ROOM - 1] = '\0';
8      /* ack testuale al client */
9  } else {
10     broadcast_line(c, line);
11 }
```

`strcmp()` riconosce il comando a inizio riga confrontando solamente i primi sei caratteri; `line + 6` sposta il puntatore oltre il prefisso ("`/nick`" o "`/join` "); `strncpy()` con limite esplicito evita overflow del buffer se l'utente invia un nome più lungo di `MAX_NAME`. A differenza di un'implementazione minimale, il client riceve una conferma testuale del cambio effettuato, invece di scoprirlo solo osservando l'effetto sui messaggi successivi.

Broadcast filtrato per stanza

```
1  for (int j = 0; j < MAX_CLIENTS; j++) {
2      if (!clients[j].active) continue;
3      if (clients[j].fd == sender->fd) continue;
4      if (strcmp(clients[j].room, sender->room) != 0) continue;
5
6      SSL_write(clients[j].ssl, out, n);
7  }
```

I tre controlli in sequenza escludono, rispettivamente: i client non più attivi, il mittente stesso (che non deve ricevere in eco il proprio messaggio) e i client che si trovano in una stanza diversa. Prima dell'introduzione di TLS l'invio avveniva con `send(..., MSG_NOSIGNAL)`: il flag evitava che una scrittura verso un client appena disconnesso terminasse il processo con `SIGPIPE`. `SSL_write()` non ha un parametro equivalente, quindi lo stesso effetto è ora ottenuto ignorando `SIGPIPE` globalmente all'avvio (`signal(SIGPIPE, SIG_IGN)`), in tutti e tre gli eseguibili.

1.4.6 Un client dedicato: `chat_client.c`

Per i test manuali si è fatto ampio uso di `netcat`, sufficiente a verificare il protocollo applicativo ma cieco rispetto a un aspetto: `netcat` non impone alcuna disciplina sull'invio delle righe, e quindi non mette mai alla prova un client che debba, come un utente reale, scrivere e ricevere messaggi contemporaneamente senza bloccarsi sull'uno o sull'altro. Per questo si è scritto un client dedicato, che multiplexa con `select()` la tastiera (`STDIN_FILENO`) e la socket verso il server — lo stesso principio del server, applicato stavolta a due soli descrittori. L'indirizzo e la porta del server sono opzionalmente configurabili da riga di comando (`./chat_client [ip] [porta]`), con `127.0.0.1` e la porta definita in `chat.h` come valori predefiniti.

Un problema inatteso: `fgets()` contro `select()`

La prima versione del client leggeva le righe da tastiera con `fgets()`, la funzione standard per la lettura riga per riga. Con un solo utente che digitava interattivamente tutto funzionava; non appena però si sono automatizzati i test con più comandi inviati in rapida successione (ad esempio `/nick` e `/join` incollati insieme, o inviati via pipe), un comando su due restava misteriosamente bloccato, inviato al server solo al momento della disconnessione.

La causa è un disallineamento tra due livelli di bufferizzazione. `fgets()` bufferizza a livello di libreria C: se in un'unica `read()` sottostante arrivano due righe complete, `fgets()` le legge entrambe ma ne restituisce una sola per chiamata, tenendo la seconda in un buffer interno invisibile a `select()`. `select()`, però, ragiona esclusivamente sul file descriptor a livello di kernel: una volta che i byte sono già stati consumati dalla `read()` interna di `fgets()`, il descrittore non ha più nulla da restituire, e `select()` non lo segnalerà di nuovo pronto finché non arrivano byte *nuovi* dal kernel — anche se una riga intera è già disponibile, sepolta nel buffer di libreria.

```
1  /* problematico: select() non "vede" il buffer interno di
   fgets() */
2  if (FD_ISSET(STDIN_FILENO, &read_fds)) {
3      char line[BUFLen];
4      fgets(line, sizeof(line), stdin);
5      send(sock_fd, line, strlen(line), 0);
6  }
```

La soluzione applicata è la stessa già adottata lato server in `feed_bytes()`: abbandonare `fgets()` e leggere lo `stdin` con `read()` grezza, facendo il framing a riga manualmente su un buffer di accumulo proprio del client.

```

1 char chunk[BUFLen];
2 ssize_t r = read(STDIN_FILENO, chunk, sizeof(chunk));
3
4 for (ssize_t i = 0; i < r; i++) {
5     if (in_len < sizeof(in_acc)) in_acc[in_len++] = chunk[i];
6     if (chunk[i] == '\n') {
7         send(sock_fd, in_acc, in_len, 0);
8         in_len = 0;
9     }
10 }

```

Con questo approccio ogni byte disponibile a livello di kernel viene consumato in un'unica `read()` per ciclo, ed eventuali più righe già accumulate vengono inviate tutte immediatamente, invece di restare in sospeso: la readiness segnalata da `select()` corrisponde sempre esattamente a ciò che viene effettivamente consumato.

Chiusura in lettura (EOF su stdin) e half-close

Un'ulteriore sottigliezza riguarda la chiusura dello stdin, ad esempio con Ctrl-D. Una volta che `read()` restituisce 0, `select()` continuerebbe a segnalare `STDIN_FILENO` pronto ad ogni ciclo successivo, poiché la condizione di EOF è persistente; il client rimuove quindi il descrittore dal `master_set` invece di ricontrollarlo, e chiude soltanto il lato in scrittura della socket con `shutdown(sock_fd, SHUT_WR)`. Terminare subito con una `break`, come in una prima versione, avrebbe rischiato di scartare una risposta del server già in transito nello stesso istante (ad esempio l'ack a un `/join` appena inviato): restando invece in ascolto sulla sola socket, il client riceve ogni messaggio residuo e termina soltanto quando è il server stesso a chiudere la connessione.

1.4.7 Cifratura del canale con TLS

Nonostante il nome, la versione iniziale del progetto non cifrava nulla: i messaggi viaggiavano in chiaro sul socket TCP. Per colmare questa distanza tra il nome e il comportamento effettivo si è integrato TLS tramite OpenSSL, sostituendo `recv()/send()` con `SSL_read()/SSL_write()` su tutti e tre gli eseguibili.

Handshake bloccante: una scelta di design deliberata

L'handshake TLS può richiedere più round-trip prima di completarsi, e gestirlo in modo pienamente asincrono dentro `select()/epoll()` avrebbe

richiesto una vera macchina a stati: durante l'handshake, `SSL_read()`/`SSL_write()` possono restituire `SSL_ERROR_WANT_READ` o `SSL_ERROR_WANT_WRITE` indipendentemente dalla direzione dell'operazione richiesta dal chiamante, e vanno riprovate solo quando il descrittore giusto (che non è detto sia quello atteso) torna pronto. Per un progetto didattico si è scelta la soluzione più semplice: eseguire `SSL_accept()` in modo **bloccante**, subito dopo `accept()` e prima di aggiungere il nuovo client al set monitorato.

```

1  int client_fd = accept(server_fd, (struct sockaddr *)&caddr,
2                               &clen);
3  /* client_fd non e' ancora nel master_set: l'handshake che
4     segue
5     * non interferisce con select(), ma il resto del server
6     resta
7     * fermo per la sua durata. */
8  SSL *ssl = SSL_new(ctx);
9  SSL_set_fd(ssl, client_fd);
10
11 if (SSL_accept(ssl) <= 0) {
12     /* handshake fallito: si scarta la connessione */
13 } else {
14     registry_add(client_fd, ssl);
15     FD_SET(client_fd, &master_set);
16 }

```

Il costo di questa scelta è esplicito: per la durata di un handshake (tipicamente pochi millisecondi in rete locale) il server, restando a singolo thread, non serve gli altri client già connessi. È lo stesso tipo di compromesso già discusso per l'array `clients[MAX_CLIENTS]` indicizzato per fd: una semplificazione accettabile alla scala di questo progetto, che andrebbe riconsiderata per un server esposto a molte connessioni concorrenti o a client volutamente lenti nell'handshake. I socket restano bloccanti per tutta la vita della connessione, come prima dell'introduzione di TLS: questo evita qualunque gestione di `SSL_ERROR_WANT_READ/WANT_WRITE` anche nel resto del programma, non solo durante l'handshake.

Un secondo disallineamento: `SSL_read()` contro `select()`

Con l'handshake bloccante funzionante, un test con due client reali ha mostrato un comportamento sconcertante: la connessione veniva accettata, ma nessun comando successivo, nemmeno il primo `/nick`, produceva una risposta. Il client restava semplicemente muto.

La causa è, ancora una volta, un buffer invisibile a `select()`, stavolta dentro OpenSSL. In TLS 1.3 il server invia di norma, subito dopo l'handshake

e senza che il client lo richieda, due messaggi `NewSessionTicket` (pensati per velocizzare una futura ripresa di sessione). `SSL_read()` li riconosce come messaggi di protocollo, non come dati applicativi, e li consuma internamente senza restituirli al chiamante: per farlo, però, deve leggere ulteriori byte dal socket sottostante, ed essendo quest'ultimo bloccante resta in attesa finché non arriva qualcosa. Se in quel momento non c'è altro da leggere, perché il client non ha ancora inviato nulla, l'unica `SSL_read()` invocata da un singolo evento di `select()` resta bloccata a tempo indeterminato, e con essa l'intero ciclo eventi: il client non torna mai a controllare la tastiera, per quanto l'utente digiti.

```

handshake completato
|
v
server invia 2x NewSessionTicket (non richiesti)
|
v
select() segnala il socket pronto --> SSL_read() UNA volta
|
v
SSL_read() consuma i due ticket, poi si blocca in attesa
di altri byte che arriveranno solo DOPO che il client scrive
|
v
il client non torna mai a select(): stallo

```

La correzione più diretta è eliminare la causa alla radice: questo progetto non ha alcun bisogno della ripresa di sessione TLS, quindi si disabilita l'invio dei ticket lato server.

```
1 SSL_CTX_set_num_tickets(ctx, 0);
```

Accanto a questa correzione mirata si è aggiunta una misura di robustezza più generale, sullo stesso principio già visto per `feed_bytes()` e per il client (Sezione 1.4.6): una singola lettura di sistema può far decifrare a OpenSSL più di un record TLS, e i byte oltre il primo restano bufferizzati internamente, invisibili a `select()/epoll()` finché non arrivano byte *nuovi* dal kernel. Dopo ogni `SSL_read()` si controlla quindi `SSL_pending()`, e si continua a leggere finché promette dati già decifrati e in attesa, prima di tornare al ciclo eventi:

```

1 feed_bytes(c, buf, (size_t)r);
2
3 while (SSL_pending(c->ssl) > 0) {
4     r = SSL_read(c->ssl, buf, sizeof(buf));
5     if (r <= 0) break;
6     feed_bytes(c, buf, (size_t)r);
7 }

```

Certificato autofirmato

Il server carica un certificato e una chiave privata autofirmati, generati in locale con `make cert (openssl req -x509 -newkey rsa:2048 ...)`; il certificato non è mai stato incluso nel repository, proprio perché contiene una chiave privata. Sul lato client, non essendoci una Certification Authority a garantire per un certificato autofirmato, la verifica è disabilitata esplicitamente con `SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL)`: una scelta accettabile per una demo in rete locale, dove il traffico è comunque cifrato e la minaccia principale (un attaccante che intercetta passivamente) è coperta, ma che espone a un attacco attivo di tipo man-in-the-middle, dato che il client non ha modo di verificare di stare effettivamente parlando con il server atteso.

1.5 Presentazione dei dati

Sono stati eseguiti i seguenti test funzionali, su entrambe le varianti e sia con `netcat` sia con `chat_client`:

- connessione di un singolo client e di più client simultaneamente;
- cambio nickname con `/nick` e stanza con `/join`;
- broadcast ricevuto solo dagli utenti della stessa stanza, verificato connettendo client in stanze diverse;
- nessun eco del messaggio al mittente;
- un messaggio applicativo spezzato artificialmente su due `send()` distinte (per verificare `feed_bytes()`), ricomposto correttamente dal server in un'unica riga — ripetuto sul canale cifrato dopo l'introduzione di TLS, con lo stesso esito;
- gestione della disconnessione (`recv()/SSL_read() ≤ 0`), con corretta rimozione del client dalla rubrica e dall'insieme monitorato;
- un tentativo di connessione in chiaro con `netcat` verso il server con TLS attivo, per verificare che l'handshake fallisca in modo pulito invece di bloccare il server o mandarlo in crash.

Il test del messaggio spezzato ha prodotto lo stesso identico risultato del messaggio inviato in un unico blocco: la riga ricomposta arriva integra al destinatario, confermando che il framing funziona indipendentemente da come il livello TCP sottostante frammenta i dati. Lo stesso identico comportamento è stato osservato avviando `chat_epoll` al posto di `chat_select`.

Il tentativo con netcat in chiaro produce, lato server, un rifiuto immediato e diagnosticabile:

```
[!] handshake TLS fallito con 127.0.0.1:60636
error:0A00010B:SSL routines:ssl3_get_record:wrong version
number
```

nessun blocco, nessun crash: il server scarta la connessione e resta pienamente operativo per tutti gli altri client.

1.6 Discussione dei risultati

Condividere la logica applicativa tra le due varianti ha permesso un confronto diretto: a parità di protocollo e di comportamento osservato, le differenze tra `select()` ed `epoll()` restano interamente confinate al meccanismo di attesa degli eventi I/O, come riassunto in Tabella 1.1.

	select()	epoll()
Complessità per ciclo	$O(\text{max_fd})$	$O(\text{fd pronti})$
Stato mantenuto da	user space (ricopiato)	kernel space
Limite connessioni	FD_SETSIZE	nessun limite fisso
Portabilità	POSIX, ovunque	solo Linux
Complessità del codice	minore	leggermente maggiore

Tabella 1.1: Confronto pratico tra `select()` ed `epoll()`

Per un progetto didattico con poche decine di client la differenza di prestazioni è trascurabile; `epoll()` diventa rilevante superate le centinaia/migliaia di connessioni simultanee, cioè proprio nello scenario C10K discusso nello stato dell'arte.

Il problema più insidioso incontrato durante lo sviluppo non è stato il multiplexing in sé, quanto il framing dei messaggi, in tre manifestazioni via via meno visibili dello stesso principio. La prima, lato server: un'implementazione che tratta ogni `recv()` come un messaggio completo appare funzionante nei test manuali con netcat, ma si rompe silenziosamente non appena un client invia dati più velocemente di quanto il server li processi. La seconda, lato client (Sezione 1.4.6): `fgets()` e `select()` bufferizzano a livelli diversi (libreria C contro kernel), e un comando può restare invisibile a entrambi finché non arriva altro traffico. La terza, la più insidiosa, dentro OpenSSL (Sezione 1.4.7): `SSL_read()` può assorbire byte di protocollo TLS senza che l'applicazione se ne accorga, e quei byte restano bufferizzati un

livello sotto quello che `select()` osserva. In tutti e tre i casi il sintomo non è mai un crash, solo un silenzio: un messaggio o un comando che semplicemente non arrivano, finché non si guarda cosa succede a livello di singola `read()` di sistema. È una lezione generale più che un incidente isolato: qualunque libreria interponga un proprio buffer tra il file descriptor e il chiamante, che sia la `libc`, `OpenSSL`, o un'altra libreria di rete, può rompere l'assunzione implicita con cui `select()` / `epoll()` vengono normalmente usati, cioè che "niente di nuovo pronto" equivalga a "niente da elaborare".

Restare in un singolo thread, come richiesto dalla consegna, ha semplificato non poco la gestione dello stato condiviso: la struttura `clients[]` viene letta e scritta da un solo flusso di esecuzione, quindi non è stato necessario alcun mutex né ci si è dovuti preoccupare di race condition. L'array `clients[MAX_CLIENTS]` indicizzato direttamente per file descriptor garantisce accesso $O(1)$, al costo di sprecare memoria se i file descriptor assegnati dal sistema sono sparsi; per le dimensioni di questo progetto è un compromesso accettabile.

```
massimilianosparta@massimilianosparta-RLEF-XX: ~/SecureChat
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ make
gcc -Wall -Wextra -O2 -o chat_client chat_client.c tls.c -lssl -lcrypto
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ make clean
rm -f chat_select chat_epoll chat_client
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ make
gcc -Wall -Wextra -O2 -o chat_select main_select.c registry.c tls.c -lssl -lcrypto
gcc -Wall -Wextra -O2 -o chat_epoll main_epoll.c registry.c tls.c -lssl -lcrypto
gcc -Wall -Wextra -O2 -o chat_client chat_client.c tls.c -lssl -lcrypto
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./chat_epoll
=== SecureChat (epoll) su porta 8080 ===
[+] client fd=5 connesso (TLS) da 127.0.0.1:45118
[+] client fd=6 connesso (TLS) da 127.0.0.1:49380
[+] client fd=7 connesso (TLS) da 127.0.0.1:49948

massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./chat_client
Connesso (TLS, TLSv1.3) a SecureChat su 127.0.0.1:8080
Comandi: /nick <nome> /join <stanza> Ctrl-D per uscire
/nick max
>> ora sei conosciuto come max
/join tennis
>> sei entrato nella stanza tennis
>> [tennis][max] ciao

massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./chat_client
Connesso (TLS, TLSv1.3) a SecureChat su 127.0.0.1:8080
Comandi: /nick <nome> /join <stanza> Ctrl-D per uscire
/nick max
>> ora sei conosciuto come max
/join tennis
>> sei entrato nella stanza tennis
ciao

massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./chat_select
bind: Address already in use
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./chat_client
Connesso (TLS, TLSv1.3) a SecureChat su 127.0.0.1:8080
Comandi: /nick <nome> /join <stanza> Ctrl-D per uscire
/nick Ugo
>> ora sei conosciuto come Ugo
ciao
```

Figura 1.3: Esecuzione di `./chat_epoll`

BIBLIOGRAFIA

- [CLRS09] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, MA, 3 edition, 2009.
- [Int81] Internet Engineering Task Force. RFC 793 - transmission control protocol (tcp) legacy. Technical report, IETF, 1981.
- [Int22] Internet Engineering Task Force. RFC 9293 - transmission control protocol (tcp). Technical report, IETF, 2022.
- [Ste03] W. Richard Stevens. *UNIX Network Programming, Volume 1: The Sockets Networking API*. Addison-Wesley, Boston, MA, 3 edition, 2003.
- [The26] The Linux man-pages project. Linux man-pages: select(2), epoll(7), socket(2), accept(2), 2026.